自动舵控制算法 C++ 代码说明文档

> **文档编号**：AR-CODE-2026-001 **版本**：V1.0 **编制日期**：2026-07-29 **对应设计文档**：《自动舵定位与控制设计文档》 §3~§5 **代码目录**：`control/include/`

---

1. 概述

本文档说明自动舵 **控制部分** 的纯算法 C++ 实现，对应设计文档中的：

- §3.2~§3.7 PD 航向控制律
- §3.6 增益调度
- §3.9 控制算法标定（Z 形试验辨识 + PD 增益初算）
- §3.10 控制参数自动优化（RLS 在线辨识 + 极点配置 + ESC）
- §4 仅航向条件下的机动功能（Leg Sequence 框架）
- §5 定位与控制联合工作流（主循环集成）

纯算法层：不包含任何硬件 IO（GPIO/CAN/UART/EEPROM），所有传感器输入与舵机输出通过函数参数传递，便于单元测试与 HIL 仿真。

---

2. 代码结构

```
control/include/
├── angle_utils.hpp          # 角度归一化与差值
├── pd_controller.hpp        # PD 航向控制器
├── gain_schedule.hpp        # 增益调度表
├── nomoto_identifier.hpp    # Nomoto 模型 RLS 在线辨识
├── auto_tuner.hpp           # 极点配置 + ESC + 稳定性监控
├── maneuver_sequencer.hpp   # 机动航段序列框架
├── param_store.hpp          # 参数存储 + CRC32
├── autopilot_core.hpp       # 核心集成（主循环）
└── tuner_debug_api.hpp      # 实船调试接口（仅调试，逐函数暴露 AutoTuner 内部机制）
```

所有文件均为 **header-only**，C++17 标准，无第三方依赖，便于嵌入 ECU。

配套实船调试工具位于 `control/tools/`（见 §3.9）：命令行 `ar_tuner_debug` 经 CMake 选项 `AR_BUILD_TOOLS`（默认 ON）构建，实时监控与端到端验收为 Python 脚本，无需构建。

3. 模块详解

3.1 angle_utils.hpp — 角度归一化

对应文档：§3.2 角度归一化

函数	说明
`normalizeAngleRad(a)`	将角度归一化到 $(-\pi, \pi]$，弧度
`normalizeAngleDeg(a)`	将角度归一化到 $(-180, 180]$，度
`angleDiffRad(ref, act)`	最小带符号差 $ref - act$，弧度
`angleDiffDeg(ref, act)`	同上，度

用途：航向误差计算必须做角度归一化，否则 359° 与 1° 的误差会被误算为 358° 而非 -2°。

```
const double e = angleDiffDeg(headingRef, headingAct);  // 落在 (-180, 180]
```

3.2 pd_controller.hpp — PD 航向控制器

对应文档：§3.2 控制律、§3.3 限幅、§3.7 抗噪与死区

控制律

$$\delta_k = K_p\, e_k + K_d\, \dot{e}_k$$

关键特性

特性	实现位置	说明
角度归一化误差	`angleDiffDeg`	§3.2
微分低通滤波	`alpha = 1/(1+τ/dt)`	§3.7 一阶低通，τ=0.2 s
误差死区	`|e| < deadzone` 输出 0	§3.7，默认 0.3°
舵角限幅	`±dMax`	§3.3，默认 ±35°

特性	实现位置	说明
舵速限幅	`ddMax·dt`	§3.3，默认 10°/s
输出量化	`round(v/quant)·quant`	§3.7，默认 0.1°

使用

```
ar::PdController::Params p;
p.Kp = 1.5; p.Kd = 0.6;
p.dt = 0.02;  // 50 Hz
ar::PdController pd(p);

double rudderCmd = pd.update(headingRef, headingAct);
```

参数说明

参数	默认值	单位	说明
Kp	1.0	—	比例增益
Kd	0.5	—	微分增益
dMax	35.0	deg	舵角限幅
ddMax	10.0	deg/s	舵速限幅
deadzone	0.3	deg	误差死区
tau	0.2	s	微分滤波时间常数
quant	0.1	deg	输出量化步长
dt	0.02	s	控制周期

3.3 gain_schedule.hpp — 增益调度

对应文档：§3.6 / §3.10.4

按船速与海况查询 PD 增益，内置默认调度表（与文档 §3.10.4 一致）：

船速区间	海况	Kp	Kd
< 4 kn	0~1	2.0	0.8
4~10 kn	0~1	1.5	0.6
4~10 kn	2~3	1.0	0.4
10~18 kn	0~1	1.2	0.5
10~18 kn	2~3	0.9	0.4
18~25 kn	0~1	1.0	0.4
18~25 kn	2~3	0.7	0.3

```
ar::GainSchedule sched;
double Kp, Kd;
sched.query(speedKn, seaState, Kp, Kd);
```

3.4 nomoto_identifier.hpp — Nomoto 模型在线辨识

对应文档：§3.9.4 Z 形试验辨识 / §3.10.3 RLS 在线辨识

模型

$$T\dot{r} + r = K\delta \;\Rightarrow\; r_k = K\delta_k - \frac{T}{\Delta t}(r_k - r_{k-1})$$

RLS 递推

$$K_k = \frac{P_{k-1}\phi_k}{\lambda + \phi_k^T P_{k-1}\phi_k}$$

$$\hat{\theta}_k = \hat{\theta}_{k-1} + K_k(y_k - \phi_k^T\hat{\theta}_{k-1})$$

$$P_k = \frac{1}{\lambda}(P_{k-1} - K_k\phi_k^T P_{k-1})$$

其中 $\theta = [K, T]^T$，$\phi_k = [\delta_k, -(r_k - r_{k-1})/\Delta t]^T$，$y_k = r_k$。

激励条件

仅当 $|\delta| > \delta_{min}$ 且（$|r| > r_{min}$ 或 $|\dot{r}| > \dot{r}_{min}$）时更新（默认 $\delta_{min} = 3^\circ$、$r_{min} = 0.5^\circ/s$、$\dot{r}_{min} = 0.2^\circ/s^2$），避免无激励时的协方差爆炸；保留 r 过零附近的高 $\dot{r}$ 瞬态，否则 T 欠定。

自降频（R1 深化）

`update(δ, r, dtNow)` 可以任意频率调用；内部累计满 `Params.beatSec`（默认 1.0 s）才做一次 RLS 拍（点采样：取拍时刻的样本）。差分 Δt 为两次 RLS 拍的真实间隔，由模块自测，与调用频率无关。旧设计"调用方按 1 s 节拍调用 + 改写 `Params.dt`"的语义陷阱已随该字段一并删除。

使用

```
ar::NomotoIdentifier id;
id.update(rudderDeg, rateDegS, dt);  // 任意频率喂入；内部自降频，满 1 s 做一次 RLS 拍
double K = id.K(), T = id.T();
```

3.5 auto_tuner.hpp — 控制参数自动优化

对应文档：§3.10

三层架构

层	方法	函数
层 1 在线辨识	RLS	`NomotoIdentifier`（独立模块，以 `IdSnapshot` 快照传入）
层 2 极点配置	闭环 ζ/ωn 反推	`AutoTuner::polePlacement`（公有静态，公式全仓库唯一出处）
层 3 性能寻优	ESC 极值搜索	`AutoTuner::tick()` 内部驱动（私有 `escStep`）

接口结构体（R1 深化新增）

```
struct IdSnapshot { bool excited; bool valid; double K; double T; };  // 辨识快照
struct TunerInput { double headingRefDeg, headingDeg, rudderDeg, speedKn, dt,
                    schedKp, schedKd; IdSnapshot id; };              // tick() 每拍输入
struct GainCommand { double Kp, Kd; };   // tick() 每拍输出（已含 ESC 扰动载波）
```

公有动作（R1 深化后仅 3 个）

动作	说明
`tick(const TunerInput&) → GainCommand`	每拍单入口：速度门控 → 无基线时跟随调度值 → 周期辨识评估（P12 漂移检查）→ 参考稳定门控 + 性能样本累积（含因果解调）→ ESC 窗口结算（稳定门 + 回退）→ 扰动注入 → 返回增益
`establishBaseline(const IdSnapshot&) → bool`	一次性评估建基线（P14 辨识试验收官 / 人工标定注入）：不做漂移检查、振荡条件以 0 把关、恒重锚；与 `tick` 共享私有 `applyEstimate_()`
`suspend()`	唯一模式生命周期钩子：停用 + 清测量状态（窗口/解调/HPF/梯度），保留基线

只读查询

`enabled()` 、 `baselineEstablished()` 、 `currentGains(Kp&, Kd&)` （去载波投影，观测增益所有权）、 `computeJ()` 、 `oscDetector()` 。

私有机制（R1 起经 tick/establishBaseline 驱动）

- 既有机制私有化： `accumulate` （性能样本 + 因果解调）、 `escDither` （扰动注入）、 `escStep` （窗口结算梯度更新）、 `revert` （回退基线）、 `rebaseEsc` （重锚清零）、 `setBaseline`
- R1 新增私有路径： `applyEstimate_` （周期评估与一次性评估的共享实现，driftGate / forceRebase 两路径）、 `settleWindow_` （原 `AutopilotCore::escTrigger` 内收）
- 从 core 迁入的状态： `curKp_/curKd_` （增益所有权，基线建立后 ESC 拥有，不再跟随调度表）、 `baseK_/baseT_` （漂移判据基准）、 `winCounter_` 、 `refAgeSec_/prevRef_` （振荡检测门控）、 `baselineEstablished_` ；P12 策略常量随迁为私有 constexpr（ `kReBaselineTol=0.25` 等）

极点配置（层 2）

$$K_p = \frac{\omega_n^2 T}{K}, \quad K_d = \frac{2\zeta\omega_n T - 1}{K}$$

```
double Kp, Kd;
AutoTuner::polePlacement(K, T, 0.85, 0.20, Kp, Kd);
```

性能指标（ITAE）

$$J = \int_0^{T_w} \left(t|e| + w_u|\delta| + w_{du}|\dot{\delta}| \right) dt$$

ESC 更新律

$$K_{p,k+1} = K_{p,k} - \eta \widehat{\partial J / \partial K_p}$$

通过对 K_p, K_d 加正弦扰动（相位错开 90° 避免耦合），解调估计梯度。

稳定性监控（§3.10.7）

```
bool ok = tuner.stabilityCheck(K, T, Kp, Kd, oscAmplitude);
// 条件：K/T 有限非退化、Kp > 0、ζ ≥ 0.6、ωn ≤ 0.5、振荡 ≤ 5°
//  （单次更新幅值 ≤ 10% 是 ESC 积分更新处私有 clampUpdate 的约束，不在本检查内）
```

安全约束

约束	阈值
单次更新幅值	≤ 10%
高速去激活	> 15 kn
稳定性四条件	全满足才接受
回退	窗口结算未过稳定门时恢复基线（私有 `revert()`，由 `tick()` 触发）

3.6 maneuver_sequencer.hpp — 机动航段序列

对应文档：§4.2 通用机动调度框架

Leg 结构

```
struct Leg {
    Mode    mode;       // HEADING / RUDDER
    double  target;     // 目标航向 或 目标舵角
    Trigger trigger;    // HEADING_REACHED / TIME / RATE / NONE
    double  threshold;  // 触发阈值
```

```
    int     next;       // 下一段索引，-1 结束
};
```

预置机动

函数	对应文档	说明
`williamson(psi0)`	§4.4	+δ_max 至 Δψ=60° → -δ_max 至 Δψ=180° → 保持
`uTurn(psi0)`	§4.5	PD 跟踪 ψ0±180°
`zigzag(psi0)`	§4.6	±δ_z 与 ±ϕ_z 交替 N 次
`circles()`	§4.3	恒舵角至累计 360°
`cloverleaf(psi0)`	§4.7	4×(转向 270° + 直航 t_leg)
`search(psi0)`	§4.8	扩张方搜，边长递增

使用

```
auto legs = ar::ManeuverSequencer::williamson(psi0);
ar::ManeuverSequencer seq;
seq.setLegs(legs);
while (!seq.done()) {
    auto cmd = seq.update(headingAct, rateAct, dt);
    // cmd.isHeading ? PD 跟踪 cmd.heading : 直接输出 cmd.rudder
}
```

3.7 param_store.hpp — 参数存储

对应文档：§3.9.8 / §3.10.9

ControlParams 结构

字段	对应 EEPROM 区块
rudderZero/Left/Right, ddotMax	区块 A
K, T	区块 B
Kp, Kd	区块 C

字段	对应 EEPROM 区块
version, timestamp, crc	元信息

CRC32

采用 IEEE 802.3 多项式 `0xEDB88320`，校验除 crc 字段外的全部内容。

IO 注入

```
ar::ParamStore store(readFn, writeFn);  // 由 BSP 注入
ar::ControlParams p;
if (!store.load(p)) { /* CRC 失败，加载默认 */ }
p.Kp = 1.5;
store.save(p);  // 自动 version+1 + CRC
```

3.8 autopilot_core.hpp — 核心集成

对应文档：§5 定位与控制联合工作流

模式

模式	说明
MANUAL	透传舵角反馈
AUTO_HEADING	航向保持（PD + 增益调度 + 自动优化）
AUTO_MANEUVER	机动序列执行
HOLD	保持当前舵角

主循环

```
ar::AutopilotCore ap;
ap.setMode(ar::Mode::AUTO_HEADING);
ap.setHeadingRef(128.5);

while (running) {
    ar::SensorInput s = readSensors();
    double rudderCmd = ap.step(s, 0.02);
```

```
    writeRudder(rudderCmd);
}
```

R1 深化：编排权移交 AutoTuner

自动优化的编排（节拍计数、ESC 窗口、速度门控、增益所有权、稳定门、回退、重锚）全部收进 `AutoTuner::tick()`；本类回归本职——模式状态机 + 机动编排 + 每拍转发。删除 `runEscTuning()`、`tunerMut()`、`escTrigger()` 与 9 个编排成员（辨识节拍计数器、ESC 窗口计数器、ESC 增益副本、基线 K/T、参考变化计时、基线标志），构造函数里的辨识器 `dt` 改写一并移除（辨识器已自降频）；新增 `setTunerParams()` 整组注入调参参数（替代暴露 mutable 引用的旧后门）。`setMode` 切出到 MANUAL/HOLD 时调 `tuner_.suspend()`（清测量状态、保留基线）。文件由 346 行收敛至 202 行。

主循环内部流程

```
step(s, dt):
  ├── MANUAL: 透传
  ├── AUTO_HEADING: headingControlStep（约 15 行）
  │    ├── 首拍以实际舵角初始化舵速限幅基准
  │    ├── 增益调度查询 Kp/Kd
  │    ├── identifier_.update(δ, r, dt)——自降频，每拍无条件喂
  │    ├── tuner_.tick({...}) → GainCommand（R1：编排全部内收）
  │    ├── pd_.setGains(g.Kp, g.Kd)
  │    └── pd_.update → 目标舵角
  ├── AUTO_MANEUVER:
  │    ├── 序列推进；辨识试验期间喂辨识器
  │    ├── 试验收官 finalizeIdentTrial()——一行 establishBaseline 映射 OK/FAILED
  │    ├── HEADING 段：增益调度查询 Kp/Kd → setGains → PD 跟踪（P13）
  │    └── RUDDER 段直接输出
  └── HOLD: 冻结上一拍舵角指令
```

3.9 tuner_debug_api.hpp — 实船调试接口（仅调试）

对应文档：`docs/实船调试接口调用文档.md`（设计文档无对应章节，调试设施，不改变任何在线行为）；工具用法的权威参考为 `control/tools/README.md`

设计意图

实船调试时需要绕开 `tick()` 的固定编排，对单一环节做激励-观测——只验极点配置数值、只打一拍载波看执行器响应、只喂误差看振荡检测读数、只把当前窗口的 J 读出来。`TunerDebugApi` 作为门面（facade）把 `AutoTuner` 的公有动作与私有机制全部摊平为独立方法：每个方法对应一个且仅

一个功能，互不影响、可任意顺序调用。访问权限由 `AutoTuner` 内的 `friend class TunerDebugApi` 授予，算法本体零改动。

使用约束（与实船安全直接相关）

约束	说明
仅调试/标定	不得接入正常航行控制回路——`force*` 系列旁路稳定门，正常路径的安全保证全部失效
行为代表性	除 `force*` 系列外，其余方法与其在 `tick()` 内部的行为逐字一致（同一私有函数、同一参数），观测结果可代表在线行为
可上 ECU	全部方法无堆分配、无 IO，可经串行/以太网命令通道驱动

公开方法分组

分组	方法	对应 AutoTuner 机制
状态观测	`snapshot()` / `currentJ()` / `oscAmplitude()`	全量快照（`Snapshot` 结构，遥测打包用）/ 当前窗口 ITAE / 实测振荡峰-峰
层 2 单功能	`polePlacement()` / `stabilityCheck()`	同名函数直转（无状态）
基线管理	`establishBaseline(K,T)` / `evaluateEstimate(K,T,osc,driftGate,forceRebase)`	公有 `establishBaseline`（试验收官语义）/ 私有 `applyEstimate_` 原样暴露
ESC 各环节	`ditherOnce` / `accumulateOnce` / `escStepOnce` / `settleWindowOnce` / `clampUpdate`	私有 `escDither` / `accumulate` / `escStep` / `settleWindow_` / `clampUpdate`
振荡检测	`oscFeed` / `oscReset`	私有 `oscDetector_` 的喂入 / 清零
回退与重锚	`revertToBaseline` / `rebaseEsc`	私有 `revert` / `rebaseEsc`
主流程与生命周期	`tick` / `suspend` / `reset`	与在线路径同一函数
参数读写	`params` / `setParams`	注意 `setParams` 复位调参器全部状态（基线清空），调用方必须知情

分组	方法	对应 AutoTuner 机制
强制注入（仅调试）	`forceGains` / `forceBaseline` / `forceEscPhase` / `forceGradient` / `forceEnabled`	直接改写当前增益 / 基线 / 载波相位 / 梯度积分器 / 速度门控标志

- `Snapshot` 字段分四类：增益与基线（`curKp/curKd`、`baselineKp/baselineKd`、`baseK/baseT`、`baselineEstablished`、`enabled`）、ESC 状态（`escPhase`、`gradKp/gradKd`、`lastJ`、`hpfY`、`demodKpSum/demodKdSum`、`demodCount`）、窗口与编排（`winCounter`、`windowSamples`、`refAgeSec`）、实测振荡（`oscAmplitude`）。
- `settleWindowOnce(valid, K, T)` 为窗末裁决原语：`escStep` 梯度结算 → 稳定门 → 接受/回退，等价 `tick()` 每窗（默认 60 s ÷ 0.02 s = 3000 拍）自动执行的那一步；辨识估计由调用方注入，便于注入测试值。
- `forceEnabled` 用于绕开速度门控：`ditherOnce` / `accumulateOnce` / `escStepOnce` 等单环节依赖 `enabled_`，单独调试这些环节时用它打开（不影响其他状态）。
- `forceBaseline` 不记录 `baseK_/baseT_`（保持 0），此后 `driftGate=true` 的漂移判据恒视为超阈值漂移、下一次评估恒重基线——保守的安全方向，但调用方应知情。

配套命令行 ar_tuner_debug（tools/tuner_debug_cli.cpp）

把 `TunerDebugApi` 的每个方法包装为逐行命令——每行一条命令，每条命令恰好调用一个功能，可经 ssh/串口终端在船上直接使用。

- 构建：`cd control/build && cmake .. && make ar_tuner_debug`（产物 `build/tools/ar_tuner_debug`）
- 输出协议：成功 `OK ...`，失败/拒绝 `ERR <原因>`；观测类输出 `key=value` 序列，便于上位机/脚本解析
- 脚本模式：仅交互终端（isatty）打印横幅与提示符；管道/重定向（`./ar_tuner_debug < cmds.txt`）下输出保持纯净可解析

命令集（`help` 可查全表）：

组	命令	功能
观测	`status` / `j` / `osc` / `params` / `setparam <名称> <值>`	全量快照 / 当前窗口 ITAE / 实测振荡峰-峰 (deg) / 参数表读 / 单参数改写
层 2	`pole <K> <T> [zeta wn]` / `check <K> <T> <Kp> <Kd> <osc>`	极点配置（缺省 ζ=0.85、ωn=0.2）/ 稳定门四条件判定

组	命令	功能
基线	`baseline <K> <T>` / `eval <K> <T> <osc> <driftGate> <forceRebase>`	一次性建基线 / 辨识评估原语
ESC	`dither <Kp> <Kd>` / `accum <e_deg> <delta_deg> [oscOk]` / `step <Kp> <Kd>` / `settle [valid K T]` / `clamp <new> <old>`	载波一拍 / 样本累积+振荡+解调 / 窗末梯度结算 / 窗末裁决 / 单次更新限幅
振荡	`oscfeed <e_deg>` / `oscreset`	只喂 / 清零振荡检测器
生命周期	`revert` / `rebase` / `suspend` / `reset`	取回基线增益 / ESC 重锚 / 挂起（清测量留基线）/ 全量复位
主流程	`tick <ref> <heading> <rudder> <speedKn> <dt> <sKp> <sKd> [exc valid K T]`	跑一拍完整在线编排，返回本拍增益
强制注入	`forcegains` / `forcebase` / `forcephase` / `forcegrad` / `forceenabled`	旁路稳定门，仅调试

关键命令细节：

- `status` 输出字段（固定顺序）：`curKp curKd baseKp baseKd baseK baseT established enabled phase gradKp gradKd lastJ hpfY demodKp demodKd demodN win samples refAge osc`。
- `setparam` 可选名称：`zeta wn escAmp escFreq escStep hpfTau lpfTau wU wDu maxUpdateFrac absMinUpdate zetaMin wnMax oscMax highSpeedKn windowSec dt`；改写即调用 `setParams`，**复位调参器状态（基线清空）**，CLI 在输出中明示该后果；未知名报 `ERR`。
- `settle` 参数全可选，缺省 `valid=1` 并注入典型船估计 K=0.2/T=8；输出裁决后的持久增益 `curKp/curKd`。
- `tick` 的辨识快照 4 参（`exc valid K T`）可选，缺省 `excited=0 valid=0`。
- `forcegains` / `forcebase` 的成功输出附带旁路警告文案。

```
$ ./build/tools/ar_tuner_debug
> pole 0.2 8              # OK Kp=1.600000 Kd=8.600000
> baseline 0.2 8          # OK established=1
> forceenabled 1          # 单独打开 ESC 门控（单环节调试前置）
> dither 1.6 8.6          # OK Kp=1.600000 Kd=9.030000（相位 0：Kp 不变、Kd +5%）
> accum 1.0 2.0 1         # OK（成本/振荡/解调各喂一拍）
> settle 1 0.2 8          # 窗末裁决：OK curKp=... curKd=...
> quit
```

实时监控 tuner_debug_monitor.py

驱动 `ar_tuner_debug` 进程（`DebugLink` 请求-响应通道，每条命令恰好一行响应、加锁线程安全），按固定周期下发 `status` 并解析 `key=value` 输出，matplotlib 分 4 个子图滚动显示：

子图	内容
增益	curKp/curKd（实线）与 baseKp/baseKd（虚线）
ESC 梯度	gradKp/gradKd（LPF 后）
窗口代价	lastJ（窗末 ITAE）
振荡/载波	实测振荡峰-峰 (°) 与载波相位 (rad)

功能要点：

- **命令转发**：监控期间在终端输入的任何调试命令原样转发给 `ar_tuner_debug`（回显打印），即"一面看曲线、一面做单环节调试"；输入 `quit` 退出。
- **`--demo` 内置激励**：`baseline 0.2 8` → `forceenabled 1` → 每 0.1 s 喂 ±2° 方波误差（周期 5 s）并打一拍载波 → 每 6 s 一次 `settle 1 0.2 8`，无实船/仿真即可看到各参数动起来，用于验证监控链路本身。
- **`--log` CSV 落盘**：每次 status 快照追加写入，事后可复查。
- **`--duration` + `--out`**：无 GUI 环境跑固定时长后保存图片退出（冒烟/归档），`PLOT_SIM_AGG=1` 强制 Agg 后端。

选项	含义	默认
`--exe PATH`	`ar_tuner_debug` 可执行文件路径	`../build/tools/ar_tuner_debug`
`--interval S`	`status` 轮询周期 (s)	0.1
`--window S`	曲线滚动时间窗 (s)	60
`--demo`	内置激励序列	关
`--log CSV`	status 快照追加写入 CSV	关
`--duration S` / `--out PNG`	固定时长后保存退出	关

实现注意：交互模式用 `FuncAnimation` 刷新，动画对象必须持有引用（`self._anim`），否则被 GC 后窗口空白（matplotlib 官方陷阱）；监控器与被调试的调参器是同一进程内实例（经管道请求-响应），船上使用时 `--exe` 指向船上命令通道的封装即可。

端到端验收 tuner_debug_accept.py

驱动 CLI 对 9 个功能组逐条命令断言预期输出，共 **49 项**，每条命令的预期数值均经手工核算；与 `test_tuner_debug_api.cpp` 互补——单元测试测 API 层，本脚本测 CLI 端到端（含参数解析与输出协议）。

组	内容
1	极点配置 / 稳定门：默认值数值、自定义 ζ/ωn、退化输入拒绝、四条件逐条违反
2	基线管理：建基线、status 一致性、漂移 10% 拒绝 / 40% 接受并重锚、退化 K 拒绝
3	ESC 各环节：门控未开载波不动、相位 0/π/2 的载波方向、600 拍累积与单通道解调、J 量级、梯度结算方向与限幅、settle 接受/回退双路径、clamp 上限与防锁死下限
4	振荡检测：±2° 方波峰-峰 ≈4°、清零
5	回退 / 重锚 / suspend / reset
6	参数读写：params 默认值、setparam 生效与复位语义、未知名报 ERR、跨组恢复默认
7	强制注入：forcegains 旁路不污染基线、forcephase 控制载波方向、forcegrad 写积分器
8	tick 主流程：高速去激活、无基线跟随调度、辨识评估建基线、建基线后不跟随调度
9	输出协议与健壮性：未知命令 / 参数不足 / 缺参均报 ERR

退出码：全部 PASS 为 0，任一 FAIL 为 1，可直接接入 CI：

```
python3 tools/tuner_debug_accept.py [--exe PATH]   # 全绿输出 "49 PASS / 0 FAIL"
```

单元测试 test_tuner_debug_api.cpp

13 个用例（220 行），围绕三条关键保证：

1. **每个功能可单独调用**：`AccumulateAndEscStepIndividuallyCallable`、`OscFeedAndReset`、`RevertRebaseSuspend` 等逐环节驱动并校验快照；
2. **与在线路径行为一致**：`PolePlacementPassThrough`、`StabilityCheckPassThrough` 与直调同名函数逐位一致；`TickThroughApiMatchesDirectTick` 对同一输入连跑 50 拍，经 API 的 `tick` 与直调 `tick` 输出逐位一致；
3. *force 确属旁路**：`ForceGainsBypassesGate_DebugOnly` 写入必不过稳定门的增益（Kp<0）仍生效，且基线不被污染。

另覆盖漂移门语义（`EvaluateEstimateDriftGateSemantics`：阈值内拒绝、超阈值接受并重锚）、载波门控与相位推进（`DitherOnceRequiresEnabledAndAdvancesPhase`）、限幅上限与绝对下限防锁死（`ClampUpdateLimitsAndAbsFloor`）、窗末裁决接受/回退双路径（`SettleWindowOnceAcceptsOrReverts`）、参数改写复位语义（`SetParamsRoundTripAndResetsState`）。

---

4. 数据流与接口

4.1 输入

```
struct SensorInput {
    double headingDeg;   // AHRS 艏向
    double rateDegS;      // AHRS 转向率
    double rudderDeg;     // 舵角反馈
    double speedKn;       // 船速
    int    seaState;      // 海况 0..3
};
```

4.2 输出

目标舵角（deg），由外部驱动层转换为 PWM/CAN 指令。

4.3 调用周期

模块	周期	说明
`pd_controller`	20 ms (50 Hz)	主控制环
`nomoto_identifier`	每拍喂入（内部自降频至 1 s）	R1：任意频率调用，累计满 `beatSec` 才做一次 RLS 拍
`auto_tuner.tick`	20 ms	R1：每拍单入口（速度门控/辨识评估/样本累积/扰动注入）
`auto_tuner` ESC 窗口结算	60 s (窗口)	R1：`tick()` 内部按 `windowSec/dt` 计数，私有 `settleWindow_` 结算
`maneuver_sequencer`	20 ms	段推进

---

5. 与设计文档的映射

设计文档章节	代码模块	关键函数
§3.2 控制律	pd_controller	`update()`
§3.3 限幅	pd_controller	内部 clip
§3.6 增益调度	gain_schedule	`query()`
§3.7 抗噪死区	pd_controller	滤波/死区/量化
§3.8 稳定性分析	auto_tuner	`stabilityCheck()`
§3.9.4 Z 形辨识	nomoto_identifier	`update()`
§3.9.5 PD 增益初算	auto_tuner	`polePlacement()`
§3.9.8 参数存储	param_store	`save()/load()`
§3.10.3 RLS 辨识	nomoto_identifier	`update()`
§3.10.4 极点配置	auto_tuner	`polePlacement()`
§3.10.5 ESC 寻优	auto_tuner	`tick()`（私有 `escStep` 结算）
§3.10.6 触发逻辑	auto_tuner	`tick()`（速度门控内收，R1）
§3.10.7 稳定性监控	auto_tuner	`stabilityCheck()`（回退经 `tick()` 内私有 `revert`）
§4.2 机动框架	maneuver_sequencer	`setLegs()/update()`
§4.3~§4.8 预置机动	maneuver_sequencer	`williamson()/uTurn()/...`
§5 联合工作流	autopilot_core	`step()`

6. 编译与测试

6.1 编译要求

- C++17
- 无第三方依赖
- header-only，直接 `#include` 即可
- 调试命令行 `ar_tuner_debug` 由 CMake 选项 `AR_BUILD_TOOLS`（默认 ON）控制，产物在 `build/tools/`；监控与验收脚本（`tuner_debug_monitor.py` / `tuner_debug_accept.py`）为 Python3 脚本（监控需 matplotlib），无需构建

6.2 单元测试

现有 103 个用例（9 个测试文件），Release + ASAN 双工具链全绿；`ar_sim --validate` 仿真验证矩阵与 R1 前逐项一致。R1 后调参器与 core 的交互经 `tick()` 行为测试断言（测试与调用方跨同一条 seam，无测试后门），如 `SelfDecimatesAtAnyCallRate`、`NoBaselinePassesScheduleGainsThrough`、`WindowSettleUpdateIsClamped`、`SettleRevertsToBaselineWhenEstimateInvalid`、`SettleAcceptsUpdateWhenEstimateHealthy`、`RebaselineClearsIntegratorState`。另有 `tools/tuner_debug_accept.py` 端到端验收（9 组 49 项断言，数值经手工核算），退出码可接 CI（见 §3.9）。

模块	测试要点	用例数
angle_utils	跨 0°/360° 边界误差	7
pd_controller	阶跃响应、限幅、死区	12
gain_schedule	跨段查询、海况限幅	7
nomoto_identifier	已知 K/T 数据收敛性、任意调用率自降频	9
auto_tuner	极点配置精度、ESC 收敛、tick 行为（基线/窗口结算/重锚）	17
maneuver_sequencer	各预置机动段推进	14
param_store	CRC 校验、版本递增	7
autopilot_core	模式状态机、机动编排、每拍转发	17

模块	测试要点	用例数
tuner_debug_api	逐函数单调用、与在线路径逐位一致、force 旁路、窗末裁决双路径	13

6.3 HIL 集成

`AutopilotCore::step()` 是唯一入口，注入 `SensorInput`、输出目标舵角，与船舶六自由度模型闭环即可进行 HIL 仿真。

---

7. 安全约束实现汇总

约束	实现位置	说明
舵角限幅 ±35°	`PdController::update`	§3.3
舵速限幅 10°/s	`PdController::update`	§3.3
误差死区 0.3°	`PdController::update`	§3.7
微分低通 τ=0.2 s	`PdController::update`	§3.7
RLS 激励门限	`NomotoIdentifier::update`	§3.10.3
单次更新 ⩽ 10%	`AutoTuner::tick`（私有 `clampUpdate`）	§3.10.7
高速去激活 > 15 kn	`AutoTuner::tick`（速度门控内收，R1）	§3.10.6
稳定性四条件	`AutoTuner::stabilityCheck`	§3.10.7
回退基线	`AutoTuner::tick`（私有 `revert`，窗口结算触发）	§3.10.7
参数 CRC32	`ParamStore::save/load`	§3.9.8
版本号递增	`ParamStore::save`	§3.10.9

约束	实现位置	说明
模式切换	`AutopilotCore::setMode`（切出自动模式时 `AutoTuner::suspend`，R1）	§5.2

---

8. 扩展接口

扩展点	接口	说明
自定义机动	`ManeuverSequencer::setLegs`	注入 Leg 序列
自定义增益调度	`GainSchedule` 构造	替换默认表
EEPROM IO	`ParamStore(ReadFn, WriteFn)`	注入 BSP 读写
性能指标权重	`AutoTuner::Params`	调整 ITAE 权重
极点配置目标	`AutoTuner::Params.zeta/wn`	按船速分段
控制周期	`PdController::Params.dt`	适配不同 ECU

— 文档结束 —